

■ 实例

在代码2.16中，`Increase<T>()`即是按函数对象形式指定的基本操作，其功能是将作为参数的引用对象的数值加一（假定元素类型`T`可直接递增或已重载操作符“++”）。于是可如`increase()`函数那样，以此基本操作做遍历即可使向量内所有元素的数值同步加一。

```
1 template <typename T> struct Increase //函数对象：递增一个T类对象
2 { virtual void operator()(T& e) { e++; } }; //假设T可直接递增或已重载++
3
4 template <typename T> void increase(Vector<T> & V) //统一递增向量中的各元素
5 { V.traverse(Increase<T>()); } //以Increase<T>()为基本操作进行遍历
```

代码2.16 基于遍历实现`increase()`功能

■ 复杂度

遍历操作本身只包含一层线性的循环迭代，故除了向量规模的因素之外，遍历所需时间应线性正比于所统一指定的基本操作所需的时间。比如在上例中，统一的基本操作`Increase<T>()`只需常数时间，故这一遍历的总体时间复杂度为 $O(n)$ 。

§ 2.6 有序向量

若向量`S[0, n)`中的所有元素不仅按线性次序存放，而且其数值大小也按此次序单调分布，则称作有序向量（**sorted vector**）。例如，所有学生的学籍记录可按学号构成一个有序向量（学生名单），使用同一跑道的所有航班可按起飞时间构成一个有序向量（航班时刻表），第二十九届奥运会男子跳高决赛中各选手的记录可按最终跳过的高度构成一个（非增）序列（名次表）。与通常的向量一样，有序向量依然不要求元素互异，故通常约定其中的元素自前（左）向后（右）构成一个非降序列，即对任意 $0 \leq i < j < n$ 都有 $S[i] \leq S[j]$ 。

2.6.1 比较器

当然，除了与无序向量一样需要支持元素之间的“判等”操作，有序向量的定义中实际上还隐含了另一更强的先决条件：各元素之间必须能够比较大小。这一条件构成了有序向量中“次序”概念的基础，否则所谓的“有序”将无从谈起。

多数高级程序语言所提供的基本数据类型都满足上述条件，比如C++语言中的整型、浮点型和字符型等，然而字符串、复数、矢量以及更为复杂的类型，则未必直接提供了某种自然的大小比较规则。采用很多方法，都可以使得大小比较操作对这些复杂数据对象可以明确定义并且可行，比如最常见的就是在内部指定某一（些）可比较的数据项，并由此确立比较的规则。这里沿用2.5.3节的约定，假设复杂数据对象已经重载了“<”和“<=”等操作符。

2.6.2 有序性甄别

作为无序向量的特例，有序向量自然可以沿用无序向量的查找算法。然而，得益于元素之间的有序性，有序向量的查找、唯一化等操作都可更快地完成。因此在实施此类操作之前，都有必要先判断当前向量是否已经有序，以便确定是否可采用更为高效的接口。

```

1 template <typename T> int Vector<T>::disordered() const { //返回向量中逆序相邻元素对的总数
2     int n = 0; //计数器
3     for (int i = 1; i < _size; i++) //逐一检查_size - 1对相邻元素
4         if (_elem[i - 1] > _elem[i]) n++; //逆序则计数
5     return n; //向量有序当且仅当n = 0
6 }

```

代码2.17 有序向量甄别算法disordered()

代码2.17即为有序向量的一个甄别算法，其原理与1.1.3节起泡排序算法相同：顺序扫描整个向量，逐一比较每一对相邻元素——向量已经有序，当且仅当它们都是顺序的。

2.6.3 唯一化

相对于无序向量，有序向量中清除重复元素的操作更为重要。正如2.5.7节所指出的，出于效率的考虑，为清除无序向量中的重复元素，一般做法往往是首先将其转化为有序向量。

■ 低效版

```

1 template <typename T> int Vector<T>::uniquify() { //有序向量重复元素剔除算法（低效版）
2     int oldSize = _size; int i = 0; //当前比对元素的秩，起始于首元素
3     while (i < _size - 1) //从前向后，逐一比对各对相邻元素
4         (_elem[i] == _elem[i + 1]) ? remove(i + 1) : i++; //若雷同，则删除后者；否则，转至后一元素
5     return oldSize - _size; //向量规模变化量，即被删除元素总数
6 }

```

代码2.18 有序向量uniquify()接口的平凡实现

唯一化算法可实现如代码2.18所示，其正确性基于如下事实：有序向量中的重复元素必然前后紧邻。于是，可以自前向后地逐一检查各对相邻元素：若二者雷同则调用remove()接口删除靠后者，否则转向下一对相邻元素。如此，扫描结束后向量中将不再含有重复元素。

这里的运行时间，主要消耗于while循环，共需迭代_size - 1 = n - 1步。此外，在最坏情况下，每次循环都需执行一次remove()操作，由2.3节的分析结论，其复杂度线性正比于被删除元素的后继元素总数。因此如图2.7所示，当大量甚至所有元素均雷同时，用于所有这些remove()操作的时间总量将高达：

$$\begin{aligned}
 & (n - 2) + (n - 3) + \dots + 2 + 1 \\
 & = O(n^2)
 \end{aligned}$$

这一效率竟与向量未排序时相同，说明该方法未能充分利用此时向量的有序性。

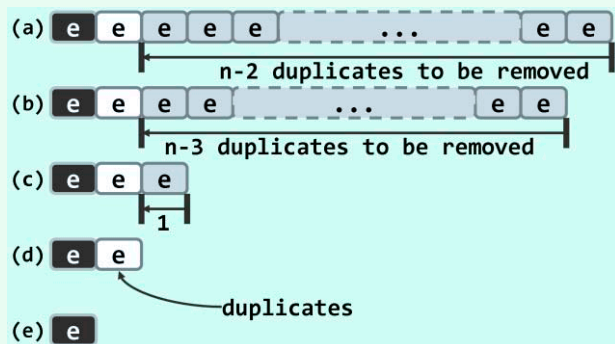


图2.7 低效版uniquify()算法的最坏情况

■ 改进思路

稍加分析即不难看出，以上唯一化过程复杂度过高的根源是，在对`remove()`接口的各次调用中，同一元素可能作为后继元素向前移动多次，且每次仅移动一个单元。

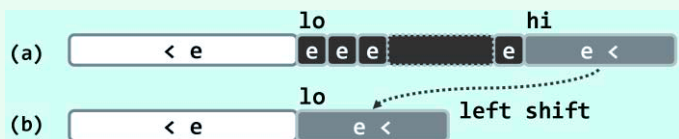


图2.8 有序向量中的重复元素可批量删除

如上所言，此时的每一组重复元素，都必然前后紧邻地集中分布。因此如图2.8所示，可以区间为单位成批地删除前后紧邻的各组重复元素，并将其后继元素（若存在）统一地大跨度前移。具体地，若 $V[lo, hi)$ 为一组紧邻的重复元素，则所有的后继元素 $V[hi, _size)$ 可统一地整体前移 $hi - lo - 1$ 个单元。

■ 高效版

按照上述思路，可如代码2.19所示得到唯一化算法的新版本。

```

1 template <typename T> int Vector<T>::uniquify() { //有序向量重复元素剔除算法（高效版）
2     Rank i = 0, j = 0; //各对互异“相邻”元素的秩
3     while (++j < _size) //逐一扫描，直至末元素
4         if (_elem[i] != _elem[j]) //跳过雷同者
5             _elem[++i] = _elem[j]; //发现不同元素时，向前移至紧邻于前者右侧
6     _size = ++i; shrink(); //直接截除尾部多余元素
7     return j - i; //向量规模变化量，即被删除元素总数
8 }

```

代码2.19 有序向量uniquify()接口的高效实现

图2.9针对一个有序向量的实例，完整地给出了该算法对应的执行过程。

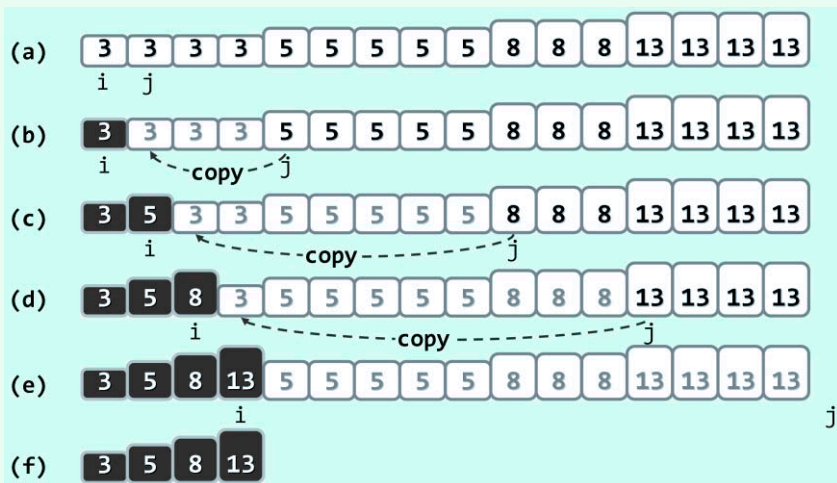


图2.9 在有序向量中查找互异的相邻元素